



Universidade Federal do Ceará
Campus Quixadá
QXD0068 – Reuso de Software
Prof. Francisco Victor da Silva Pinheiro

Trabalho 2

Serviço Reutilizável (SOA/Microserviço) ou Mini-LPS

GABRIEL DIAS DE LIMA

MARCELO SILVA DE LIMA

Quixadá-CE
2025

Sumário

1. Introdução
2. Descrição da Solução
3. Tecnologias e Padrões Utilizados
4. Implementação
5. Avaliação da Reusabilidade
6. Conclusão
7. Referências

1. Introdução

A análise de algoritmos é uma das áreas mais pesquisadas entre cientistas da computação. Através dela se pode trabalhar melhor a execução e eficiência de algoritmos, bem como explorar seu potencial para diversas áreas do conhecimento e para aplicações na vida real, como indústria, interconectividade, educação e saúde.

No entanto, a depender do algoritmo analisado, um cientista da computação pode ser travado pelas limitações de hardware e/ou dificuldade de solucionar o problema. Problemas da classe NP-Difícil, por exemplo, mostram-se particularmente áduos devido ao desconhecimento de soluções em tempo polinomial para suas resoluções. Isso dificulta a análise desses algoritmos, uma vez que a máquina a ser utilizada para sua análise pode sofrer flutuações de performance, tornando a análise da execução menos precisa.

O problema é particularmente evidente no caso específico da comparação entre dois algoritmos, em que, a depender da inconstância ao executar o programa, pode-se obter resultados que não equivalham entre si, visto a possibilidade de um destes algoritmos ser executado em um ambiente mais propício que o outro, levando a resultados enviesados. Por isso, faz-se necessária a aplicação de execução paralela e/ou concorrente entre dois programas sob comparação, permitindo, assim, que ambos sejam analisados em condições similares.

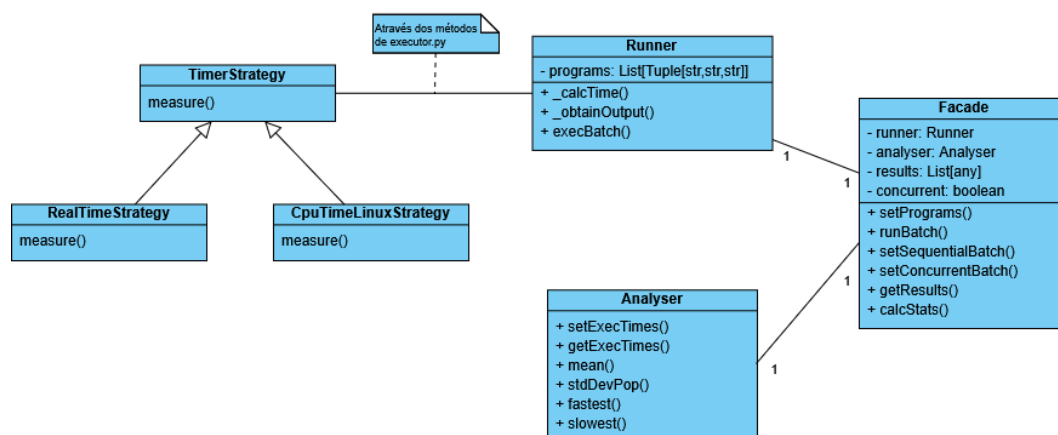
Portanto, ferramentas de execução paralela entre diferentes algoritmos se apresentam com um potencial considerável para o reuso. Uma ferramenta capaz de execução paralela, e que apresente estatísticas relevantes para a comparação, pode poupar tempo considerável dos cientistas, que não precisarão implementar tal artefato por conta própria e/ou analisar continuamente os programas em execução para obter uma figura clara da eficiência dos algoritmos.

O objetivo do trabalho é implementar uma biblioteca reutilizável em Python, capaz de executar programas de diferentes linguagens de programação em *batch*, a fim de auxiliar cientistas da computação na tarefa de comparação de algoritmos, outputs e/ou linguagens, através de uma interface de fácil uso e configuração. Nela é possível obter os tempos de execução concorrente e sequencial, calcular estatísticas, extrair o output dos programas executados e retornar possíveis mensagens de erros, no caso de código quebrado.

2. Descrição da Solução

A solução apresenta uma biblioteca que permite a execução em *batch* de algoritmos, o que permite que, apesar da sua implementação em uma linguagem específica, possa-se rodar aplicações de linguagens de programação distintas. Através dela é possível não só executá-los, como também obter estatísticas e métricas relevantes para o comparativo. Para isso, são disponibilizadas opções de métricas de tempo (tempo de CPU e cronológico), além da possibilidade de execução concorrente, através de corrotinas, como também a execução sequencial, entregando grande flexibilidade ao usuário para realizar seus experimentos.

O domínio principal da aplicação, por consequência, é a ciência da computação, especificamente para a análise e comparação de algoritmos. Seu intuito é facilitar o trabalho de cientistas da computação, poupando tempo que pode ser reinvestido em sua pesquisa.



O diagrama mostra os relacionamentos entre as classes presentes na biblioteca. No total, são implementadas seis classes ao todo, das quais três são utilizadas na aplicação do Strategy para os métodos de medição de tempo. O usuário se comunica através da Facade, que simplifica o acesso das funcionalidades ao usuário.

3. Tecnologias e Padrões Utilizados

A biblioteca foi implementada na linguagem Python. A escolha foi influenciada por ser uma linguagem de fácil interpretabilidade e bem difundida, o que facilita o trabalho dos cientistas da computação para operá-la, mesmo que não tenham grande expertise com Python. Recomenda-se a utilização do Python versão 3.10 ou superior. Além disso, todo o código foi implementado visando a funcionalidade em um sistema operacional Linux, não sendo recomendada a execução em plataformas Windows (mais detalhes são dados na conclusão).

A solução utiliza diversos módulos da própria linguagem em sua implementação. Destacam-se o module “subprocess”, que é responsável por rodar comandos e programas externos no código; “time”, módulo encarregado das tarefas relacionadas à medição de tempo no projeto; “os”, usado para gerenciar diretórios; realizar operações de arquivo e execução de comandos; “multiprocessing”, um módulo que permite o uso paralelo de processos separados, sendo essencial para a realização da execução simultânea dos algoritmos; e o “typing”, fornecedor de dicas de tipo para os métodos e classes da linguagem, incluído para tornar o código mais explícito e legível.

```
class TimerStrategy(ABC):
    @abstractmethod
    def measure(self, command: List[str]) -> float:
        pass

class RealTimeStrategy(TimerStrategy):
    def measure(self, command: List[str]) -> float:
        start = time.time()
        sp.run(
            command,
            stdout=sp.PIPE,
            stderr=sp.PIPE)
        end = time.time() - start
        return end

class CpuTimeLinuxStrategy(TimerStrategy):
    def measure(self, command: List[str]) -> float:
        subProcess = sp.Popen(
            command,
            stdout=sp.PIPE,
            stderr=sp.PIPE)
        _, _, rusage = os.wait4(subProcess.pid, 0)
        cpuTime = rusage.ru_utime + rusage.ru_stime
        return cpuTime
```

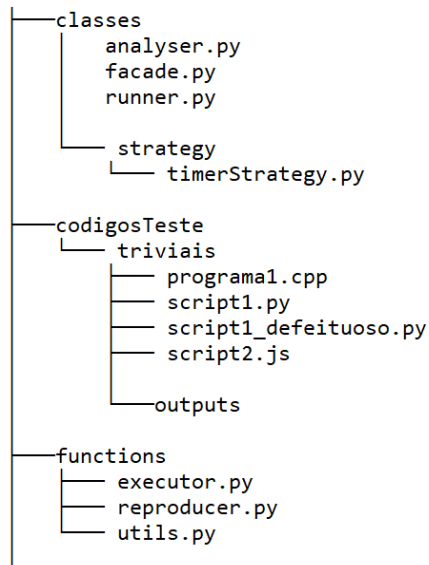
Dois padrões foram empregados para o projeto. O primeiro foi o Strategy. Tal padrão foi usado para gerenciar a escolha entre obter o tempo de CPU ou cronológico dos algoritmos. Para isso, implementa-se uma classe TimerStrategy, que implementa o método abstrato “measure()”, que recebe uma lista de strings “command” e retorna um float ao usuário. Duas outras classes, RealTimeStrategy e CpuTimeLinuxStrategy, herdam da classe principal e reimplementam o método measure() de acordo com as peculiaridades de cada maneira de se medir o tempo. No caso, RealTimeStrategy oferece a medição no “real time”, enquanto “CpuTimeLinuxStrategy” mede através do “CPU time”. O “real time” refere-se ao tempo total gasto para executar uma

determinada tarefa, do início da execução do tempo ao fim da marcação. Enquanto isso, o “CPU time” é o tempo gasto especificamente pela CPU para executar o código.

Por fim, uma função independente `timer_selector()` retorna uma das duas classes herdadas a partir da escolha do usuário por tempo de CPU ou cronológico. Tal função é chamada na `cronometrarSubprocess()` da `executor.py`, onde seu retorno é dividido entre o objeto recebido (`timer`) e o seu tipo em uma string (`timerType`).

Outro padrão importante foi o Facade. Por conta da diversidade de classes e funções do projeto, verificou-se que havia a necessidade de simplificar a interface que o usuário tem acesso, de tal modo que ele possa configurar seu experimento de modo mais explícito. Através dele se pode passar todos os parâmetros necessários para a comparação, extrair as estatísticas do desempenho dos algoritmos e personalizar o modo de execução, como, por exemplo, definir se a execução deve ser sequencial ou paralela.

4. Implementação



O projeto se organiza em duas pastas principais. A primeira é a de classes, onde se guardam as classes utilizadas no código. É aqui onde ficam as classes do timerStrategy.py anteriormente descritas, especificamente dentro da pasta strategy. Analyser, em analyser.py, é responsável por gerenciar as operações que envolvam os resultados obtidos. Nela é onde se controla os tempos de execução e se obtêm as estatísticas.

```
class Analyser:
    def __init__(self, execTimes=[]):
        self.execTimes: Tuple[str, float] = execTimes

    def setExecTimes(self, execTimes):
        self.execTimes = execTimes

    def getExecTimes(self):
        return self.execTimes

    def mean(self) -> float:
        return statistics.mean([time[1][1] for time in self.execTimes])

    def stdDevPop(self) -> float:
        return statistics.pstdev([time[1][1] for time in self.execTimes])

    def fastest(self) -> Tuple[str, float]:
        return sorted(self.execTimes, key=lambda elem: elem[1][1])[0]

    def slowest(self) -> Tuple[str, float]:
        return sorted(self.execTimes, key=lambda elem: elem[1][1])[-1]
```

A segunda classe é a Runner, presente no arquivo Runner.py. Ela fica encarregada de controlar a execução do batch em si, junto dos diferentes parâmetros que podem ser definidos pelo usuário. Em seu construtor, ela recebe uma lista de programas a serem executados. Ela controla o cálculo do tempo (de CPU ou cronológico), a obtenção do output dos programas executados em batch e a captura de signals. Também é através da classe Runner que os códigos são rodados de modo concorrente ou sequencial.

A classe Facade, no arquivo facade.py, é a interface que o usuário interage ao utilizar o código. Através dela é possível definir os principais parâmetros das classes anteriores, abstraindo, assim, sua utilização, tornando a tarefa de configurar o experimento mais simples ao cientista da computação. Nela se pode definir os principais parâmetros da execução, como, por exemplo, definir os programas a serem utilizados através da setPrograms(). As funções setConcurrentBatch() e setSequentialBatch(), por sua vez, ficam responsáveis por definir o modo de execução, sendo eles concorrente e sequencial, respectivamente. No caso da setConcurrentBatch(), o método de execução é por concorrência, onde múltiplas tarefas são alternadas para dar a impressão de que são executadas simultaneamente, ou seja, de modo mais próximo de uma execução paralela. O setSequentialBatch(), por sua vez, define o método de execução como sequencial, ou seja, cada programa será executado individualmente um após o outro.

A função runBatch() é o método principal da classe, sendo o responsável por informar o objeto da classe Runner presente no Facade os parâmetros escolhidos pelo usuário. Os resultados obtidos dessa execução podem ser obtidos através da função getResults(). Por fim, também é possível obter as estatísticas dos resultados obtidos, por meio de calcStats(), de tal modo que o usuário possa escolher exatamente quais métricas ele gostaria de obter a partir dos resultados obtidos.

```
from classes.runner import Runner
from classes.analyser import Analyser

class Facade:
    def __init__(self, programs=None):
        self.runner = Runner(programs)
        self.analyser = Analyser()
        self.results = None
        self.concurrent = True
        self.dataType = None # "Time" se o tempo for obtido, "Output" se a saída for obtida

    def setPrograms(self, programs):
        self.runner.programs = programs

    def runBatch(self,
                 cpuTime=False,
                 realTime=False,
                 captureOutput=False,
                 captureSignal=False) -> None:

        if (realTime and cpuTime):
            raise ValueError("Combinação inválida de parâmetros: só é possível escolher uma métrica de tempo.")
        if (realTime or cpuTime):
            self.dataType = "TIME"
        elif (captureOutput or captureSignal):
            self.dataType = "OUTPUT"

        self.results = self.runner.execBatch(self.concurrent, cpuTime,
                                             realTime, captureOutput, captureSignal)

    def setSequentialBatch(self):
        self.concurrent = False

    def setConcurrentBatch(self):
        self.concurrent = True

    def getResults(self):
        return self.results

    def calcStats(self, mean=False, stdDevPop=False, slowest=False, fastest=False):
        if self.results is None:
            raise TypeError("Nenhum dado obtido. Você rodou o 'runBatch'?")
        if self.dataType != "TIME":
            raise TypeError("Operação inválida. Não é possível obter estatísticas numéricas com outputs.")

        stats = []
        self.analyser.setExecTimes(self.results)
        if mean:
            mean_ = self.analyser.mean()
            stats.append(mean_)
        if stdDevPop:
            stdDevPop_ = self.analyser.stdDevPop()
            stats.append(stdDevPop_)
        if slowest:
            slowest_ = self.analyser.slowest()
            stats.append(slowest_)
        if fastest:
            fastest_ = self.analyser.fastest()
            stats.append(fastest_)
        return tuple(stats)
```

A segunda pasta de código é a functions, onde são guardadas as funções genéricas utilizadas em outras partes do código. Os métodos relacionados à execução do código (chamados em no runBatch() da classe Runner) ficam em executor.py. As funcionalidades presentes incluem a cronometragem dos subprocessos, o tratamento dos comandos de execução, a obtenção do output dos programas e a obtenção dos signals fornecidos pelos métodos. Em reproducer.py, por sua vez, guardam-se as funções relativas aos paths e destinos, bem como a criação de arquivos de output onde são armazenados o tempo de execução ou as saídas dos códigos fornecidos. Por fim, em utils.py ficam as funções initVariables(), responsável por definir valores padrões para algumas variáveis na execução, e a queue2List(), que trata os valores obtidos da execBatch() do Runner para serem retornados posteriormente.

A execução através da interface é simples e prática. Existem dois arquivos de teste disponíveis: *exemploSimples.py* e *utilizacaoReuso.py*. Para rodar, basta usar o comando `python3 <nomedoarquivo>`, utilizando um sistema operacional Linux ou MacOS.

A seguir, apresentamos o exemplo incluído em *utilizacaoReuso.py*, de uma implementação de um comparador de algoritmos de sorting, especificamente o Bubblesort, Mergesort, Quicksort e o Insertionsort, implementados nas linguagens Python e JavaScript. No total, são 8 programas distintos a serem comparados. Para a execução deste exemplo, presente em *example.py*, faz-se

necessária a presença das bibliotecas matplotlib e numpy para plotar o gráfico com os resultados obtidos da execução em batch.

```
def main():

    programs = [
        ("test/python_code/quicksort_py.py", "codigosTeste/results/outputs", "python3"),
        ("test/python_code/mergesort_py.py", "codigosTeste/results/outputs", "python3"),
        ("test/python_code/insertionsort_py.py", "codigosTeste/results/outputs", "python3"),
        ("test/python_code/bubblesort_py.py", "codigosTeste/results/outputs", "python3"),

        ("test/js_code/quicksort_js.js", "codigosTeste/results/outputs", "node"),
        ("test/js_code/mergesort_js.js", "codigosTeste/results/outputs", "node"),
        ("test/js_code/insertionsort_js.js", "codigosTeste/results/outputs", "node"),
        ("test/js_code/bubblesort_js.js", "codigosTeste/results/outputs", "node")
    ]

    interface = Facade()

    interface.setPrograms(programs)
    interface.setConcurrentBatch()
    interface.runBatch(cpuTime= True)

    #print(interface.getResults())

    py, js = parse_sort_results(interface.getResults())
    plot_sort_benchmarks(py, js)

    interface.runBatch(captureOutput=True)
    results = interface.getResults()
    print(results)

    save_outputs_to_file(results, "outputs.txt")
```

Primeiramente, define-se uma lista de tuplas dos programas, contendo suas origens, o path de destino dos outputs e, por fim, seus comandos de execução (“node” para o JavaScript, “python3” para Python, etc). Após isso, o usuário pode instanciar um objeto Facade(), o qual pode receber os programas através do método setPrograms(). Posteriormente, faz-se necessário definir o tipo de execução, ou seja, se os programas serão executados sequencialmente ou paralelamente. Para isso, utiliza-se um dos métodos entre setConcurrentBatch() e setSequentialBatch() presentes no Facade, já anteriormente descritos.

ExecBatch() roda o programa depois de tudo definido. Os resultados obtidos de tal execução podem ser acessados através de getResults(), e através disso o usuário pode trabalhá-los da forma que desejar. No exemplo das imagens, uma função auxiliar divide os resultados obtidos em suas respectivas linguagens. Após isso, é possível rodá-los em outra função que extrai tais resultados para plotá-los em um gráfico.

Depois de plotado o gráfico, o programa executa um novo runBatch(), dessa vez com captureOutput como verdadeiro. Isso serve para obter a saída das funções, que é armazenada na variável results. Tal variável é passada para uma função, junto de outro parâmetro contendo o outputs.txt, onde as saídas ficam armazenadas.

5. Avaliação da Reusabilidade

Existe uma larga gama de possibilidades para a aplicação do componente reusável. A execução em batch permite sua utilização, por exemplo, em experimentos acadêmicos, como para projetos relacionados a projeto e análise de algoritmos, teoria dos grafos, matemática computacional e outras disciplinas que envolvam algum grau de avaliação entre algoritmos. Também é possível reaplicar o código em projetos que envolvam a avaliação da consistência de outputs, avaliando se algoritmos distintos para uma mesma tarefa geram as mesmas saídas. Outra possibilidade é a implementação de um serviço em nuvem para a comparação desses algoritmos, onde o usuário envia os programas através da rede até um servidor que, por sua vez, utiliza-se da implementação da biblioteca para gerar as estatísticas e resultados dos experimentos.

Outras soluções não-reutilizáveis envolvem a implementação de código que permita a execução do paralelismo, da medição de tempo e da obtenção de estatísticas. Caso o cientista da computação deseje comparar algoritmos em diferentes linguagens, cada uma destas tarefas terá que ser replicada nos contextos individuais destas linguagens, isto é, cada conjunto de experimentos realizados em uma linguagem específica terá que possuir meios próprios de se aplicar a execução concorrente e medi-la.

Já com a biblioteca implementada o desenvolvedor tem muito mais liberdade para aplicar diferentes algoritmos de diferentes linguagens em um único ambiente. Além disso, a possibilidade de execução sequencial e concorrente, além das estatísticas fornecidas, permite que o usuário possa abstrair boa parte do seu trabalho de comparação entre os programas quando comparado a métodos que não envolvam o reuso.

6. Conclusão

Com isso, conclui-se que a implementação consegue simplificar a execução e comparação de código de diferentes linguagens com a utilização do reuso. Consegue-se configurar experimentos com diversos parâmetros úteis para realizar a medição do tempo gasto com a execução de algoritmos, além de ser possível obter as saídas obtidas de tais execuções. Um benefício claro é a economia de tempo para o analista, visto que não há necessidade de se criar múltiplas classes e funções em cada linguagem experimentada dedicadas para analisar suas performances. Além disso, a possibilidade de retornar os *sinais* de interrupção do sistema operacional, como *segmentation faults*, também auxilia em encontrar possíveis erros nos algoritmos.

O maior desafio enfrentado durante a implementação foi tratar a quantidade de parâmetros utilizados na comunicação entre classes e funções. Diversas tipagens específicas e estruturas incompatíveis causaram percalços na implementação dos códigos. Uma outra entrave se deu por conta da estrutura do projeto, que ficou gradativamente mais complexa com o tempo. Isso forçou um rearranjo dos caminhos das pastas e arquivos do projeto, o que, felizmente, reduziu a verbosidade dos *imports* e facilitou a legibilidade do código.

Outrossim, uma vez que a biblioteca se comunica com vários recursos do baixo nível do desenvolvimento de software, como *buffers* de saída e registros de erro, foi necessário um estudo detalhado acerca das especificidades dos principais sistemas operacionais. Desse modo, foi possível constatar que o *Windows* apresentou sérias dificuldades no que tange a obtenção dos *tempos de CPU*, uma vez que alguns programas não conseguiam terminar a sua execução. Isso se deve pois, diferente de sistemas operacionais baseados em UNIX, como Linux e MacOS que mantêm os processos filhos vivos até que a execução do processo pai termine, os processos filhos do Windows terminam assim que eles chegam ao fim.

Em última instância, isso fez com que fosse impossível obter o tempo de CPU de subprocessos que terminassem antes da obtenção do seu identificador, PID, que era necessário para obtenção da métrica de *tempo de CPU*. Em virtude dessa impossibilidade, foi decidido não portar a biblioteca para ambientes windows, porém isso pode ser facilmente driblado pela utilização do WSL (windows subsystem for Linux) para a execução da biblioteca.

7. Referências

- <https://docs.python.org/3/library/multiprocessing.html>
- <https://docs.python.org/3/library/subprocess.html>